Defiende el servidor

MLOps 2026-II · Universidad Externado de Colombia · Docente: Julián Zuluaga · 30 de octubre de 2026

Integrantes Miguel Melo (grupo A) · Daniel Santiago Rincón Gamba (grupo B)

Código github.com/Daniel-RGS/defiende-el-servidor-unsw-nb15 (tag `entrega-final`)

Modelos huggingface.co/Daniel-RGS/defiende-el-servidor-unsw-nb15 · v1 `c4ed6901110c506b99438df76933c779fc9e737c` · v2 `d71ac6a63d9a80ecc7af6c13601676a71b925da3`

Construimos un clasificador binario de conexiones de red (0 normal, 1 ataque) con UNSW-NB15 y, sobre todo, una cadena de versiones que permite recuperarlo: el código en Git, los datos en DVC, los experimentos en MLflow y los modelos en Hugging Face. La clase positiva es ataque en todas las métricas. El objetivo no fue la métrica más alta, sino poder explicar de qué datos y experimento sale cada modelo y volver a usarlo desde cero.

1. Datos y limpieza

Usamos las particiones publicadas `UNSW_NB15_training-set.csv` (175.341 filas) y `UNSW_NB15_testing-set.csv` (82.332 filas), con 45 columnas cada una. Sus SHA-256 coinciden con las huellas del anexo de la guía. No hay faltantes. El objetivo es `label`; excluimos `label`, `attack_cat` (revela la respuesta) e `id` (identifica filas y su orden está asociado a la clase: los primeros 47.911 `id` son todos normales), y dejamos 42 predictoras: 39 numéricas y 3 categóricas (`proto`, `service`, `state`).

Contamos duplicados solo con las 42 predictoras. En entrenamiento hay 101.040 grupos distintos; 9.748 se repiten y suman 74.301 filas más allá de la primera aparición. Hay 229 grupos con etiquetas contradictorias (940 filas): la misma entrada aparece como normal y como ataque. Decidimos conservarlos y no resolverlos por mayoría, porque eliminarlos o relabelarlos cambiaría la distribución; lo que sí garantizamos es que ningún grupo quede partido entre entrenamiento y validación.

Creamos entrenamiento y validación solo desde el archivo de entrenamiento, con `GroupShuffleSplit` (20 % para validación, semilla `20261027`), usando como grupo el hash de las 42 predictoras. Resultado: 139.850 filas de entrenamiento y 35.491 de validación, con cero grupos compartidos. La asignación fila a fila (`split_assignments.csv`) quedó fija para todos los experimentos. El preprocesamiento (imputación por mediana o moda, escalado y one-hot con `handle_unknown="ignore"`) está dentro de un único `Pipeline` de scikit-learn que solo se ajusta con entrenamiento y se guarda completo junto al estimador.

Del test solo auditamos esquema y cruces: 6.886 filas (8,4 %) repiten grupos de entrenamiento y 1.655 (2,0 %) grupos de validación. Ese solapamiento puede favorecer las métricas de test. En DVC quedaron dos versiones: los CSV recibidos (`data/raw/*.dvc`, commit `5b646e2`) y los datos preparados con sus splits (`data/prepared.dvc`, commit `be164fb`).

2. Experimentos

Corrimos seis configuraciones de tres familias con los mismos splits; cada una tiene un cambio identificado respecto a una configuración anterior. Todas quedaron en MLflow con hipótesis, parámetros, semilla, versión de datos, commit, métricas, matriz de confusión y el pipeline completo como artefacto. La tabla usa validación y umbral 0,5.

Configuración	Cambio e hipótesis	Recall	Precision	FPR	Macro-F1	Run ID
logistic_baseline	Referencia lineal, C=1	0,9881	0,9246	0,1712	0,9247	`69f2df8e`
logistic_balanced	class_weight balanced: cambiar recall/FPR	0,9519	0,9513	0,1035	0,9243	`07e38640`
logistic_regularization	C=0,1: más regularización, mejor macro-F1	0,9890	0,9221	0,1775	0,9228	`854b4a4a`
tree_baseline	Árbol, profundidad 12: relaciones no lineales	0,9784	0,9427	0,1264	0,9352	`ca5f347a`
tree_balanced	Árbol balanceado: menos FPR, menos recall	0,9287	0,9775	0,0455	0,9294	`8dcf18fb`
random_forest	100 árboles balanceados, profundidad 18: menos varianza	0,9477	0,9756	0,0504	0,9415	`dcdbd1d1`

Fuente: `evidencias/experimentos.csv` y `evidencias/mlflow/<run_id>/`. Los run ID completos están en ambos archivos.

La regularización no ayudó (macro-F1 bajó), y balancear la logística bajó el FPR pero no subió el recall: dos hipótesis que no se cumplieron. Los árboles superaron a las logísticas en macro-F1, y el balanceo en árboles produjo el intercambio esperado (FPR 0,046 a cambio de recall 0,929). El bosque mejoró al árbol balanceado en macro-F1 y en recall. Sin los 940 registros contradictorios, las métricas cambian menos de 0,005 (`metricas-sensibilidad.csv`), así que esas filas no explican las diferencias entre modelos, aunque sí imponen un techo: dos filas idénticas con etiquetas opuestas no pueden acertarse a la vez.

Los denominadores cero se manejan con `zero_division=0` en precision, recall y macro-F1, y el FPR vale 0 si no hay negativos reales; en nuestros datos no ocurrió. Reprodujimos dos corridas en otro equipo: el bosque dio métricas idénticas y la logística cambió 4 de 35.491 predicciones por diferencias numéricas del optimizador entre plataformas (`evidencias/reproduccion-experimento.json`).

3. Elección de umbral

En seguridad pesa más dejar pasar un ataque que una falsa alarma, pero un FPR alto satura al equipo que responde. Fijamos el criterio el 27 de octubre, antes del barrido: maximizar el recall de ataque con un FPR de validación de 5 % como máximo; en empates, macro-F1 y luego precision. Lo escribimos después de ver las métricas a 0,5 de validación (nunca las de test) y lo declaramos así en `criterio-seleccion.json`. Aplicamos la misma rejilla (0,00 a 1,00 cada 0,01) a los seis modelos.

Modelo	Mejor umbral factible	Recall	Precision	FPR	Macro-F1
random_forest	0,51	**0,9463**	0,9762	0,0490	0,9410
tree_balanced	0,49	0,9291	0,9773	0,0458	0,9296
tree_baseline	0,70	0,9253	0,9796	0,0410	0,9287
logistic_balanced	0,63	0,8947	0,9748	0,0491	0,9038
logistic_baseline	0,78	0,8926	0,9751	0,0484	0,9026
logistic_regularization	0,78	0,8882	0,9749	0,0485	0,8995

Fuente: `evidencias/umbral-candidatos.csv` y `umbral-sweep-all.csv`. Solo validación.

El bosque con umbral 0,51 obtuvo el mayor recall factible. Su margen frente al límite es pequeño (0,0490 contra 0,05), lo que anotamos antes de evaluar test como un riesgo.

4. Versiones v1 y v2

Campo	v1	v2
Modelo y umbral	Logística (`logistic_baseline`), 0,50	Random forest balanceado, 0,51
Tag Git	`v1` = `dc23c56` (config. `configs/v1.json`)	`v2` = `b453a7f` (config. `configs/v2.json`)
Commit de entrenamiento	`7c35ad5b8734ab5fb8f5b100d7284ba01838a01c` (las seis corridas)	
Datos DVC	`data/prepared.dvc`, md5 `3d25c4d9…`, commit `be164fb`	
Run MLflow	`69f2df8e614b4efc8cea9ea98e442368`	`dcdbd1d174e442039826cfb6d38ccb99`
SHA-256 del pipeline	`3cf85c94…deefbff`	`5c187304…f83f74342`
Validación: recall / precision / FPR / macro-F1	0,9881 / 0,9246 / 0,1712 / 0,9247	0,9463 / 0,9762 / 0,0490 / 0,9410
Matriz `[[TN, FP], [FN, TP]]`	`[[9414, 1944], [288, 23845]]`	`[[10801, 557], [1296, 22837]]`

v1 es la primera solución funcional: la referencia logística con umbral por defecto. v2 cambia la familia de modelo y el umbral, con base en los experimentos. En validación, v2 baja las falsas alarmas de 1.944 a 557 (FPR de 0,171 a 0,049) y sube la precision; a cambio, los ataques no detectados pasan de 288 a 1.296 (recall de 0,988 a 0,946). Aceptamos ese costo porque v1 marcaría como ataque a uno de cada seis flujos normales, algo inmanejable en operación. Ambas revisiones de Hugging Face contienen el pipeline completo, `config.json` con el umbral, `requirements.txt` y la model card con propósito, datos, métricas, limitaciones, uso y referencias a Git, DVC y MLflow.

5. Recuperación

Hicimos la prueba en una carpeta y un entorno nuevos, con caché de Hugging Face vacía y sin la caché DVC original (`scripts/recuperacion_limpia.sh`): clonamos el commit, instalamos las dependencias fijadas, bajamos el respaldo del almacenamiento DVC desde Drive, verificamos su SHA-256, hicimos `dvc pull` y comprobamos las huellas de los datos originales y preparados. Luego descargamos v1 y v2 por su SHA completo, verificamos el hash de cada `pipeline.joblib` y predijimos sobre 20 filas fijas de validación. Con tolerancia 1e-12, las predicciones coincidieron con las guardadas antes de publicar: diferencia máxima 0 en v1 y 2,2e-16 en v2, con etiquetas iguales. Una prueba de mutación (alterar una etiqueta de referencia) hizo fallar la comparación, como debía. Log, entradas, salidas y hashes están en `evidencias/recuperacion/final/`.

6. Test final

Evaluamos una sola vez, el 30 de octubre, la revisión congelada de v2 (`d71ac6a6…`, tag `freeze-pre-test` sobre el commit `eaaa2e6` del 27 de octubre) sobre las 82.332 filas del test, con la regla `y_pred = 1` si `score_attack ≥ 0,51`. El script verifica los hashes del modelo y del test y se niega a repetir la evaluación.

	Recall	Precision	FPR	Macro-F1	Accuracy	Matriz `[[TN, FP], [FN, TP]]`
Validación	0,9463	0,9762	0,0490	0,9410	0,9478	`[[10801, 557], [1296, 22837]]`
Test	0,9649	0,8791	0,1626	0,9053	0,9076	`[[30983, 6017], [1591, 43741]]`

El recall se mantuvo e incluso subió, pero el FPR se triplicó y superó con amplitud el límite de 5 % que guió la selección. Las particiones publicadas no tienen la misma composición: el test tiene 44,9 % de tráfico normal frente a 31,9 % en entrenamiento, y en el test una fracción mayor de esos flujos normales recibe puntajes por

encima del umbral. No reajustamos el umbral ni repetimos la evaluación, porque eso habría convertido el test en validación. Las métricas se recalculan desde `predicciones-test.csv` con `src/recalculate_test_metrics.py` y coinciden con `metricas-test.json`. Actualizamos la model card de v2 con este resultado mediante un commit de solo documentación, que conserva la revisión evaluada y el hash del artefacto.

7. Limitaciones

- El umbral elegido no generalizó su FPR al test; en operación habría que recalibrarlo con tráfico normal reciente.
- Cerca del 10,4 % del test repite grupos de entrenamiento o validación, lo que puede inflar sus métricas.
- Las métricas pesan cada fila, incluidas las repeticiones, y 940 filas de validación tienen etiquetas contradictorias.
- El criterio de umbral se escribió después de ver las métricas a 0,5 de validación, aunque antes del barrido y sin mirar el test.
- UNSW-NB15 es un conjunto histórico de laboratorio; estos resultados no demuestran eficacia en una red actual.
- El remote principal de DVC (DagsHub) requiere credenciales; la recuperación sin ellas depende del respaldo en Drive.
- Reentrenar la logística en otra plataforma puede cambiar unas pocas predicciones; los modelos entregados se recuperan por SHA, no se reentrenan.

Referencias

Moustafa, N. y Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. *Military Communications and Information Systems Conference (MilCIS)*. UNSW Canberra.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825-2830.

Mitchell, M. et al. (2019). Model Cards for Model Reporting. *FAT* '19*.

Documentación de DVC, MLflow Tracking y Hugging Face Hub (descarga por revisión), consultada en octubre de 2026.

Uso de IA: Daniel usó Codex y Miguel usó Claude Code como asistentes; la declaración detallada está en `entrega.md`.